

PRODUCT REQUIREMENTS DOCUMENT

Warehouse-Backed Grocery Self-Checkout & Restocking System

Problem Statement

A grocery store keeps its item stock in a CSV file instead of only in memory, since this file is shared with the store's warehouse. The store has a limited number of self-checkout kiosks, and stock steadily runs down as customers buy items, with no automatic way to flag low stock or restock it until a delivery van arrives.

Build a terminal-based system that manages self-checkout and van restocking using this inventory file as the single source of truth. Customers should be assigned to a free kiosk (or queued if all are busy), have their cart checked against available stock, and receive a loyalty discount that is worked out automatically from how many times they have shopped at the store. A delivery van should be able to restock items by adding quantities back into the same file. Every change must be saved back to the file, and items running low should be flagged.

The objective is to keep checkout fast and the inventory file always accurate, while correctly applying discounts and letting van restocks and customer purchases update stock without conflicts or negative quantities.

Interface

Run the application using: `python main.py` (reads and writes `inventory.csv` and `customers.csv` in the same folder; `customers.csv` is created automatically after the first completed purchase).

```
-----
CO-OP GROCERY - SELF CHECKOUT SYSTEM
-----

Kiosk Status:
K1   Busy  (C101, Basic)
K2   Free
K3   Free
K4   Free

Queue: (empty)

Inventory Snapshot:
ITEM_ID NAME          QTY    PRICE
I001   Milk 1L        40    Rs 60
I002   Bread          25    Rs 40
I003   Eggs (12pk)    15    Rs 90
I004   Rice 5kg        8    Rs 350
I005   Cooking Oil 1L   3    Rs 180  [LOW STOCK]
I006   Sugar 1kg       30    Rs 45
I007   Tea Powder     18    Rs 120
I008   Detergent      12    Rs 150

1. Start Checkout
2. Complete Checkout
3. Process Van Delivery
4. View Inventory
5. Exit

Enter choice:
```

Inputs

Start Checkout

- Customer ID (no membership tier is typed in; it is worked out automatically)

Complete Checkout

- Kiosk ID of a busy kiosk
- One or more (Item ID, Quantity) pairs, ended by typing 'done'

Process Van Delivery

- Van Manifest ID
- One or more (Item ID, Quantity Delivered) pairs, ended by typing 'done'

Loyalty tiers (worked out automatically)

Visit number	Tier	Discount
1 (first purchase)	Basic	0%
2 to 9	Silver	5%
10 and above	Gold	10%

The visit number is the number of completed purchases the customer has made before, plus one for the visit in progress. Discounts are rounded up to the next whole rupee.

Outputs

- Kiosk status (with the customer and tier on busy kiosks), the waiting queue, and the inventory snapshot, shown before every menu.
- Checkout receipt: customer ID, visit number, tier, items purchased, subtotal, loyalty discount, total.
- Restock summary: item-by-item old quantity -> new quantity after a van delivery.
- LOW STOCK flag on any item at or below the minimum threshold (5 units).
- Validation messages for invalid items, insufficient stock, invalid quantities, and invalid menu choices.
- Exit message, after saving the inventory and customer files back to disk.

Functional Requirements

FR ID	Requirement
FR-01	Load inventory (item ID, name, quantity, price) from inventory.csv at startup, and load each customer's visit count from customers.csv (an empty history if that file does not exist yet).
FR-02	Display kiosk status, the waiting queue, and an inventory snapshot before every menu.
FR-03	Start Checkout: read a Customer ID and work out the customer's visit number and loyalty tier automatically from their stored visit count.
FR-04	Assign the lowest-numbered free kiosk to the customer and mark it Busy.
FR-05	Add the customer to a first-come-first-served queue, showing their position, if every kiosk is busy.
FR-06	Complete Checkout: let the user pick a busy kiosk and enter that customer's cart as (Item ID, Quantity) pairs until 'done'.
FR-07	Deduct each accepted quantity from inventory immediately and save the updated inventory to inventory.csv.
FR-08	Calculate the subtotal from item prices and quantities purchased.
FR-09	Apply the tier discount (Basic 0%, Silver 5%, Gold 10%), rounded up, and calculate the total.
FR-10	Display a complete receipt after a successful purchase.
FR-11	After a successful, non-empty purchase, increase the customer's visit count by one and save it to customers.csv.
FR-12	Release the kiosk once checkout is complete, and assign it to the customer at the front of the queue, if anyone is waiting.
FR-13	Process Van Delivery: accept a manifest ID and (Item ID, Quantity Delivered) pairs, add the quantities to existing stock, and save to inventory.csv.
FR-14	Display a restock summary showing the old and new quantity of each restocked item.
FR-15	Flag any item whose quantity is at or below the minimum stock threshold as LOW STOCK.
FR-16	Support a menu option to view the current inventory at any time.
FR-17	On Exit, save the inventory and customer files and display a goodbye message.

Test Cases

Test Case	Input / Situation	Expected Output
TC-01	Start the application with a populated inventory.csv.	Inventory loads correctly.
TC-02	The main menu is displayed.	All five options are listed: Start Checkout, Complete Checkout, Process Van Delivery, View Inventory, Exit.
TC-03	Checkout: Milk 1L x2 (stock 40, price Rs 60).	Subtotal = Rs 120; stock reduced to 38.
TC-04	A customer on their 10th visit (Gold) buys Rice 5kg x1, subtotal Rs 350.	10% discount = Rs 35; total = Rs 315.
TC-05	A first-time (Basic) customer buys Rice 5kg x1.	No loyalty discount is applied.
TC-06	Checkout requesting more units than available stock.	Rejected with an insufficient-stock message; stock unchanged.
TC-07	A customer starts checkout when all kiosks are busy.	Customer is added to the queue instead of a kiosk.
TC-08	Checkout completes successfully.	The kiosk used is released and shown as Free again.
TC-09	Van delivery: Cooking Oil 1L +50 (stock 3).	Stock updates to 53 and is saved to inventory.csv.
TC-10	Van delivery for an Item ID not in inventory.	Rejected with an invalid item message.
TC-11	An item's stock is at or below the minimum threshold.	Item is flagged as LOW STOCK in the inventory display.
TC-12	Restart the application after a previous session's restock.	Inventory reflects the values last saved to inventory.csv, not the original defaults.
TC-13	User selects Exit.	Inventory is saved and Goodbye! is displayed.

Edge Cases

EC ID	Input / Situation	Expected Behaviour
EC-01	User enters an invalid menu option.	Display Invalid choice. and show the menu again.
EC-02	Cart contains an Item ID that does not exist in inventory.	Reject that line with an invalid item message; the rest of the cart continues.
EC-03	Requested checkout quantity exceeds current stock.	Reject that line with an insufficient-stock message; inventory unchanged for that item.
EC-04	Requested quantity is zero, negative, or not a number.	Display Invalid quantity value. and ask for the quantity again.
EC-05	Complete Checkout is chosen when no kiosk has an active customer, or a kiosk ID with no customer is entered.	Display a no-active-session message and return to the menu.
EC-06	All kiosks are busy when a new customer arrives.	Customer is placed into a queue rather than being turned away.
EC-07	A kiosk becomes free while customers are waiting.	The kiosk is assigned to the customer at the front of the queue (first-come-first-served).
EC-08	Van delivery specifies an Item ID not present in inventory.	Reject that line of the manifest with an invalid item message.
EC-09	Van delivery quantity is zero, negative, or not a number.	Display Invalid quantity value. and ask for the quantity again.
EC-10	inventory.csv is missing or unreadable at startup.	Display a clear error message and exit cleanly instead of crashing.
EC-11	Two operations change the same item's stock one after the other.	Each operation finishes (update and save) before the next begins, keeping the file consistent.
EC-12	An item's stock is exactly equal to the minimum threshold.	Item is flagged as LOW STOCK (the threshold is inclusive).
EC-13	'done' is typed immediately at checkout with an empty cart.	No receipt is printed, the kiosk is released, and the visit is not counted toward the customer's tier.